

CyberOps Test — Complete Revision Cheat Sheet

Covering the Cyber Security Operations Process Cycle (Phases 01–07) with NIST 2.0 mapping

THE PROCESS CYCLE AT A GLANCE

Phase	Name	NIST 2.0	One-Liner
01	IT Systems Landscape	IDENTIFY	Know what you're protecting
02	Aggregation & Normalisation	PROTECT	Standardise and consolidate all security data
03	SOC & SIEM Functions	DETECT	Monitor, correlate, and identify threats in real-time
04	Service Management Integration	GOVERN	Bridge security alerts to operational response
05	Workflow Automation	PROTECT + DETECT + RESPOND	Automate repetitive tasks, reduce human error
06	Incident Response	RESPOND	Contain, mitigate, and remediate active threats
07	Remediation, Recovery, Resilience	RECOVER	Restore, learn, and harden for the future

Key structural points: Phase 07 feeds back into Phase 01 → the cycle is **continuous**, not linear. Phase 05 (Workflow Automation) underpins the entire cycle (spans three NIST functions). 'Remediation – Recovery – Resilience' runs along the top AND bottom of the diagram — it is ever-present.

PHASE 01 — IT SYSTEMS LANDSCAPE (NIST IDENTIFY)

- Document the **entire IT environment**: systems, tools, processes, infrastructure
- Establishes the scope of what needs monitoring and protection
- Feeds every subsequent phase — **you cannot secure what you don't know exists**
- Includes: asset inventories, network maps, system dependencies, service catalogues

PHASE 02 — AGGREGATION & NORMALISATION (NIST PROTECT)

Normalisation (making data consistent)

Convert raw logs from different sources into a single uniform schema so they can be analysed together.

Key standardisations:

- Timestamps → **ISO 8601** format (e.g. 2024-02-02T10:15:23Z), preferably UTC
- Severity → unified scale: **info** → **low** → **medium** → **high** → **critical**
- Event types → lowercase with underscores (e.g. authentication_failure)
- IP addresses and ports → consistent formatting

Required fields in every normalised event:

- normalised_timestamp, event_type, source_type, severity

Optional fields: fill with empty string if unavailable, never omit. Source-specific data → additional_details. **Always preserve the original raw log data.**

Four source types to know:

Source	Raw Format	Example Event Type
Windows Event Log	CSV	authentication_failure (EventID 4625)
Firewall Log	JSON	network_traffic

Source	Raw Format	Example Event Type
IDS Alert	Plain text	security_alert (e.g. SQL injection)
VPN Access Log	Pipe-delimited	vpn_connection / vpn_failure

Windows EventID mappings: 4625 = auth failure, 4624 = auth success, 4688 = process execution, 5140 = network access

Normalisation steps:

- 1. Identify source log formats (structure, fields, special characteristics)
- 2. Create field mapping strategy (source fields → normalised template)
- 3. Standardise common fields (timestamps, severity, event types, IPs)
- 4. Handle data type conversions (numbers to strings, booleans, derived fields)
- 5. Implement normalisation rules (apply mappings, handle nulls, preserve originals)
- 6. Validate normalised output (all required fields present, formats correct)

Aggregation (combining normalised data for analysis)

The 8-step aggregation pipeline:

- 1. **Collect** normalised events from all sources
- 2. **Create combined chronological timeline** (sort by timestamp, assign sequence IDs)
- 3. **Group related events** by common attributes (source IP, username, event type)
- 4. **Create event correlations** (identify cause-and-effect between events)
- 5. **Generate summary statistics** (event counts, failure rates, severity distribution)
- 6. **Build incident timelines** (chronological event chain for each incident)
- 7. **Document attack patterns** (characteristic sequences, indicators, timeframes)
- 8. **Generate comprehensive reports** (executive summaries + technical details)

Aggregation transforms isolated events into **actionable security intelligence** by revealing patterns invisible when looking at individual sources separately.

Common challenges:

- Data quality: inconsistent timestamps, missing values, duplicates
- Volume: large data sets, real-time processing demands, storage limits
- Correlation: false correlations, time sync issues, gaps in event sequences
- Standardisation: different naming conventions, severity scales, categorisation

Tools: Security Onion (all-in-one VM), ELK Stack (Elasticsearch/Logstash/Kibana), Graylog

PHASE 03 — SOC & SIEM FUNCTIONS (NIST DETECT)

SIEM = Security Information and Event Management

Two core components:

- **SIM** (Security Information Management) — long-term storage and historical analysis of log data
- **SEM** (Security Event Management) — real-time monitoring, event correlation, and alerting

What a SIEM does:

- Gathers data from across the IT infrastructure (network devices, security tools, servers, apps)
- Normalises incoming data into a standardised structure
- Applies **correlation rules** to identify patterns indicating security incidents
- Uses **threat intelligence** to contextualise events and identify **IOCs** (Indicators of Compromise)

Primary SIEM outcomes:

- 1. Comprehensive visibility across infrastructure
- 2. Rapid threat detection via real-time monitoring

- 3. Regulatory compliance through systematic log management
- 4. Accelerated incident response via automated workflows
- 5. Unified security monitoring platform

Correlation Rules

Logical conditions programmed into a SIEM that connect different security events to identify potential incidents.

Brute Force Detection Rule (Week 4 Lab):

IF: 5+ failed login attempts | FROM: same source IP | WITHIN: 120-second window | ON: SSH service

THEN: Generate alert = 'Brute Force Detected'

Why these parameters: Same IP = correlation key | Time window = distinguishes attacks from normal | Threshold (5) = balances detection vs false positives | SSH = common attack vector

- **Sliding window:** For each failed login, check if enough additional failures occurred within the time threshold
- **Correlation vs Aggregation:** Correlation examines **relationships** (same IP, time proximity); aggregation just counts totals
- Simple counting fails to capture: **the temporal relationship between events** (timing patterns and clustering)

The False Positive Problem

False positive: An alert triggered by benign activity that appears malicious but isn't actually a threat.

IP	Activity	Classification	Fidelity
192.168.1.100	Brute force attack	True Positive	—
10.0.0.5	Monitoring server checking SSH every 2 min	False Positive	—
Overall	2 alerts, 1 real	50% accuracy	Low

Why false positives matter: Alert fatigue (analysts ignore alerts), wasted time, missed real threats, unnecessary automated responses.

Identify 10.0.0.5 as legitimate: Username is 'monitor' not 'admin', regular 2-min interval (automated), RFC 1918 internal IP, documented monitoring system.

Fix: Whitelisting — excludes known-legitimate sources from alerting, improving accuracy from 50% → 100%. It **acknowledges** known-good sources, it does NOT ignore them entirely or block them.

Key SIEM terminology:

- **Alert fidelity** — accuracy of alerts (high = minimal false positives)
- **Alert lifecycle** — detection → triage → investigation → remediation → closure
- **Behavioural analytics** — establish normal patterns, flag deviations (UBA, EBA, NBA)
- **Risk-based alerting** — priority based on asset criticality + potential impact
- **Data enrichment** — enhancing raw data with threat intel, asset DBs, user directories
- **Baseline configuration** — documented 'normal' behaviour for anomaly detection
- **Data retention** — policies for how long data is stored (active vs archived)

PHASE 04 — SERVICE MANAGEMENT INTEGRATION (NIST GOVERN)

Purpose: Bridge the gap between security monitoring and operational response.

Three core components:

- **1. Incident Management Integration** — links alerts to ITSM workflows, automates ticket creation, tracks full lifecycle
- **2. Service Level Agreements (SLAs)** — documented commitments on response times by severity
- **3. Continuous Service Improvement** — measure effectiveness, identify optimisation areas, implement feedback

SLA Response Times:

Severity	Response Time	Assigned Team
CRITICAL	15 minutes	Incident Response
HIGH	1 hour	Security Operations Centre
MEDIUM	4 hours	Network Security
LOW	8 hours	IT Support

Severity Calculation Algorithm (Week 5 Lab — VERY testable!)

Step 1: Start with baseline severity from the alert

Step 2: Check event count: 10+ → CRITICAL | 5–9 → HIGH | 3–4 → MEDIUM | <3 → LOW

Step 3: Check timing: Off-hours (before 9 AM or after 5 PM) → **escalate one level** | Business hours → no change

Step 4: Check whitelist: Known-good IP → **always downgrade to LOW** regardless of event count

Why off-hours escalation? Fewer analysts available to respond quickly.

Why whitelisting? Acknowledge known-good sources generating expected activity — NOT to completely ignore or block.

Worked example:

Alert: 6 failed logins from 192.168.1.100 at 09:15 AM

Step 1: Baseline = HIGH | Step 2: Count=6 → HIGH | Step 3: 09:15=Business hrs → no change | Step 4: Not whitelisted

→ Final: HIGH severity, SOC team, 1 hour SLA

Exam scenario:

Q: 12 failed logins at 3:00 AM — severity and SLA?

A: Event count 12 → CRITICAL. Time 3:00 AM = off-hours → escalate, but already CRITICAL (max). **CRITICAL severity, 15-minute SLA, Incident Response team.**

Key KPIs:

Metric	Business Value
MTTR (Mean Time to Resolve)	Faster response = less damage from attacks
SLA Compliance	Demonstrates consistent service delivery
Team Distribution	Ensures no team is overwhelmed
Escalation Rate	Tracks severity trends over time

If escalation rate jumps from 10% → 40% = more CRITICAL incidents occurring, indicating **increased threat activity**.

Key terminology:

- **CMDB** — Configuration Management Database (IT infrastructure + relationships repository)
- **OLA** — Operational Level Agreement (internal IT group support agreement)
- **KPI** — Key Performance Indicator
- **RTO** — Recovery Time Objective (target time to restore after disruption)
- **BIA** — Business Impact Analysis
- **Known Error** — a problem analysed but not yet resolved
- **Service Catalogue** — documented inventory of available security services + SLAs

Operating WITHOUT a SOC:

- Security handled through standard IT service desk, using MSSPs, automated tools, cloud services
- Challenges: limited expertise, slower response, gaps in 24/7 coverage, less threat correlation
- Compensating measures: enhanced logging, regular assessments, IR retainers, awareness training

PHASE 05 — WORKFLOW AUTOMATION (NIST PROTECT + DETECT + RESPOND)

Purpose: Automate repetitive tasks, accelerate incident response, reduce human error.

Key concepts:

- **Playbook** — predefined automated action sequence for a specific security scenario
- **Runbook** — detailed procedural document (may include manual steps)
- **Orchestration** — coordinating multiple automated processes/tools for complex workflows
- **SOAR** — Security Orchestration, Automation and Response (the platform category)

The Alert-to-Ticket Workflow (Week 5 lab):

1. Alert — correlation rule triggers → **2. Calculate** — severity logic → **3. SLA** — response time → **4. Ticket** — create & structure → **5. Route** — assign & notify

This pattern = **orchestration** (coordinating multiple automated steps in sequence)

Integration methods:

- **API Integration** — connects security tools, enables automated data exchange
- **Event-Driven Automation** — triggers on specific events, executes predefined responses
- **Conditional Logic** — decision-based workflow routing, context-aware automation

Success metrics:

- **Automation Rate** — % of tasks automated vs manual
- **Time Savings** — reduction in time on routine tasks
- **Error Reduction** — decrease in mistakes/incomplete actions
- **Response Time** — improvement in incident response speed

Tools:

- Enterprise SOAR: Splunk Phantom, IBM Security SOAR, Cortex XSOAR
- Open Source: TheHive (case management), WALKOFF (NSA-developed)
- ITSM: ServiceNow Security Operations, Jira Service Management
- Scripting: Python, PowerShell | Config Mgmt: Ansible, Puppet

PHASE 06 — INCIDENT RESPONSE (NIST RESPOND)

Execute actions to contain, mitigate, and remediate identified threats. Minimise impact. Restore normal operations.

The Complete Alert Lifecycle (5 stages):

Stage	Actions	Key Point
1. Detection	Correlation rule triggers, alert generated with metadata, severity assigned	Pattern match initiates the process
2. Triage	Review details, check whitelists/known-good sources	Determine TRUE POSITIVE vs FALSE POSITIVE
3. Investigation	Full log context, check for successful_login after failures, identify affected systems, gather forensic evidence	Was the attack successful?
4. Response	Block IP at firewall, reset credentials, isolate systems, create ticket in TheHive/ServiceNow	Automated or manual actions
5. Documentation & Closure	Record actions, update correlation rules, add to threat intel (MISP), close case	Feed lessons back into the cycle

PHASE 07 — REMEDIATION, RECOVERY, RESILIENCE (NIST RECOVER)

Four aims:

- 1. Restore affected systems 2. Eliminate threats 3. Prevent recurrence 4. Ensure resilience + continuity

Week 7 Lab — Remediation Planning Structure:

Part	Name	Priority	Key Actions
1	Containment	FIRST	Isolate systems, block IPs/domains, preserve evidence
2	Eradication	NEXT	Reset credentials, clean up systems, apply patches
3	Recovery	THEN	Verify backups, rebuild/restore, prioritise (P1=Critical, P2=Important, P3=Normal), comms plan
4	Resilience	ONGOING	Detection rules for each attack stage, preventive controls (MFA, segmentation, DLP), policy updates, lessons learned

Eight actionable objectives:

- 1. Implement recovery plans 2. Root Cause Analysis 3. Integrate lessons learned 4. Update policies
- 5. Enhance monitoring 6. Add preventive controls 7. Test recovery procedures 8. Train staff

The Feedback Loop: Phase 07 connects back to every earlier phase — updates landscape docs (01), enhances aggregation (02), expands SOC monitoring (03), improves service management (04), automates new workflows (05).
Cyber security is a continuous cycle.

Case Study: GameForge Studios

- **Attack:** Exploited third-party auth vulnerability → accessed dev servers + player account data
- **Detection:** Unusual database queries flagged by monitoring
- **Recovery:** Rebuilt from clean baselines, verified backups, emergency patches, credential resets
- **Resilience:** Dedicated security team, bug bounty, pen testing, third-party assessments, security champions
- **Result:** Prevented similar attacks within 6 months, security became a competitive advantage

LAB SKILLS & LINUX COMMANDS

Ubuntu log locations:

- /var/log/syslog — general system messages
- /var/log/auth.log — authentication events
- systemd journal — binary logs via journald

Key commands:

```
sudo tail -n 20 /var/log/syslog - View last 20 syslog entries
sudo tail -n 20 /var/log/auth.log - View last 20 auth entries
journalctl -n 20 - View last 20 journal entries
sudo tail -n 100 /var/log/auth.log > auth_sample.log - Extract sample
```

The lab pipeline: Collection → Parsing → Normalisation → Visualisation → Analysis
This mirrors what enterprise SIEM platforms do at massive scale.

- **Python log parsing** uses regex to extract structured fields; normalisation creates dictionaries with standardised keys
- **Docker** provides isolated, consistent environments — NOT because it's faster, required for SIEM, or detects threats
- Understanding correlation logic helps you **configure tool-specific rules, debug issues, and explain your SIEM**

KNOWLEDGE CHECK ANSWER KEY

Question	Correct Answer
How can personnel analyse logs in different formats?	By standardising different log formats for consistent analysis
Why extract only last 100 lines?	To reduce file size and focus on recent, relevant events
What does <code>parse_auth_log()</code> do?	Extracts failed SSH logins and structures them consistently
Advantage of automated log parsing?	Quick pattern detection across thousands of events humans couldn't process
What does normalisation achieve?	Creates dictionaries with standardised keys so different formats analysed together
How does the lab prepare for SIEM?	Log parsing, normalisation, and visualisation are foundational SIEM concepts
Why use Docker?	Isolated, consistent environment with identical data and dependencies
What does simple counting fail to capture?	Temporal relationships between events (timing patterns and clustering)
Correlation vs aggregation?	Correlation examines relationships (same IP, time proximity); aggregation counts totals
What does the sliding window do?	Checks if enough failures occurred within the time threshold per login
Why alerts on both IPs?	Both had 5+ failed SSH attempts within 120-second threshold
What is a false positive?	Alert triggered by benign activity that appears malicious but isn't a threat
How did whitelisting help?	Excluded known-legitimate sources, accuracy 50% → 100%
Which phase determines true vs false positive?	Triage — analysts check whitelists and known patterns
Why is understanding correlation logic valuable?	Configure tool-specific rules, debug issues, explain your SIEM
12 failed logins at 3:00 AM — severity?	CRITICAL, 15-minute SLA — high count + off-hours
Why escalate off-hours incidents?	Fewer analysts available to respond quickly
Purpose of whitelist in severity calculation?	Acknowledge known-good sources generating expected activity
What workflow pattern does the code implement?	Orchestration: coordinating multiple automated steps in sequence
Escalation rate 10% → 40%?	More CRITICAL incidents = increased threat activity

NIST CSF 2.0 FUNCTIONS QUICK REFERENCE

Function	Purpose	Phase(s)
IDENTIFY	Know your assets, risks, environment	Phase 01
PROTECT	Safeguards to ensure service delivery	Phase 02, 05
DETECT	Identify security events in real-time	Phase 03, 05
GOVERN	Align security with business objectives	Phase 04
RESPOND	Take action on detected incidents	Phase 05, 06
RECOVER	Restore capabilities and resilience	Phase 07

GLOSSARY OF KEY TERMS

Term	Definition
IOC	Indicator of Compromise — forensic evidence suggesting malicious activity
SIEM	Security Information and Event Management — central intelligence platform

Term	Definition
SIM	Security Information Management — long-term storage and historical analysis
SEM	Security Event Management — real-time monitoring and alerting
SOC	Security Operations Centre — team monitoring and responding to threats
SOAR	Security Orchestration, Automation and Response
SLA	Service Level Agreement — documented response time commitments
OLA	Operational Level Agreement — internal IT group support agreement
CMDB	Configuration Management Database — IT infrastructure repository
KPI	Key Performance Indicator
MTTD	Mean Time to Detect
MTTR	Mean Time to Respond / Mean Time to Resolve (same acronym, different meanings)
RTO	Recovery Time Objective — target time to restore service after disruption
BIA	Business Impact Analysis — assessment of disruption effects
MSSP	Managed Security Service Provider
RCA	Root Cause Analysis
MISP	Malware Information Sharing Platform (threat intelligence)
CSI	Continuous Service Improvement
Playbook	Predefined automated action sequence for specific scenarios
Runbook	Detailed procedural document (may include manual steps)
Orchestration	Coordinating multiple automated processes together
Correlation Rule	Logical condition connecting events to identify incidents
Alert Fidelity	Accuracy of alerts (high = few false positives)
False Positive	Alert triggered by benign activity mistaken for a threat
Whitelist	Known-good sources excluded from alerting
Data Enrichment	Enhancing raw data with external context (threat intel, asset DBs)
Normalisation	Converting varied log formats into a consistent standard schema
Aggregation	Combining normalised events to reveal patterns and relationships
Baseline	Documented standard of normal behaviour for anomaly detection
Known Error	A problem that has been analysed but not yet resolved